CMSC424: Database Design

Module: Query Processing and Optimization

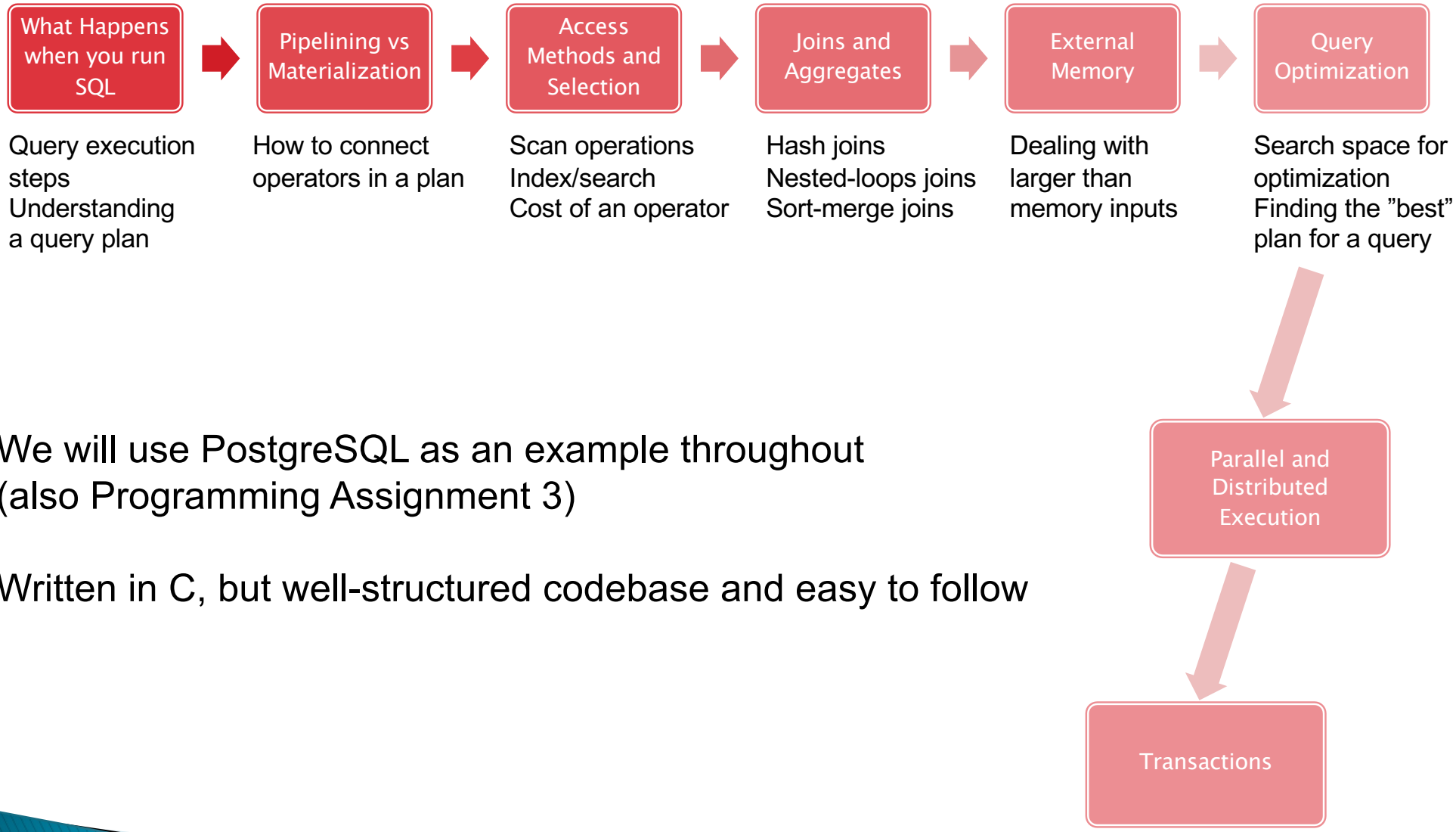
Instructor: Amol Deshpande
amol@cs.umd.edu

Database Implementation

- Shifting into discussing the internals of a DBMS
  - How data stored? How queries/transactions executed?

- Some of the Topics:
  - Storage: How is data stored? Important features of the storage devices (RAM, Disks, SSDs, etc)
  - Tuple Organization: How are tuples laid out
  - Indexes: How to quickly find specific tuples of interest (e.g., all 'friends' of 'user0')
  - Query processing: How to execute different relational operations? How to combine them to execute an SQL query? How to do this in a parallel setting?
  - Query optimization: How to choose the best way to execute a query?
  - Transactions: How to support the ACID properties? In a distributed setting?

Plan



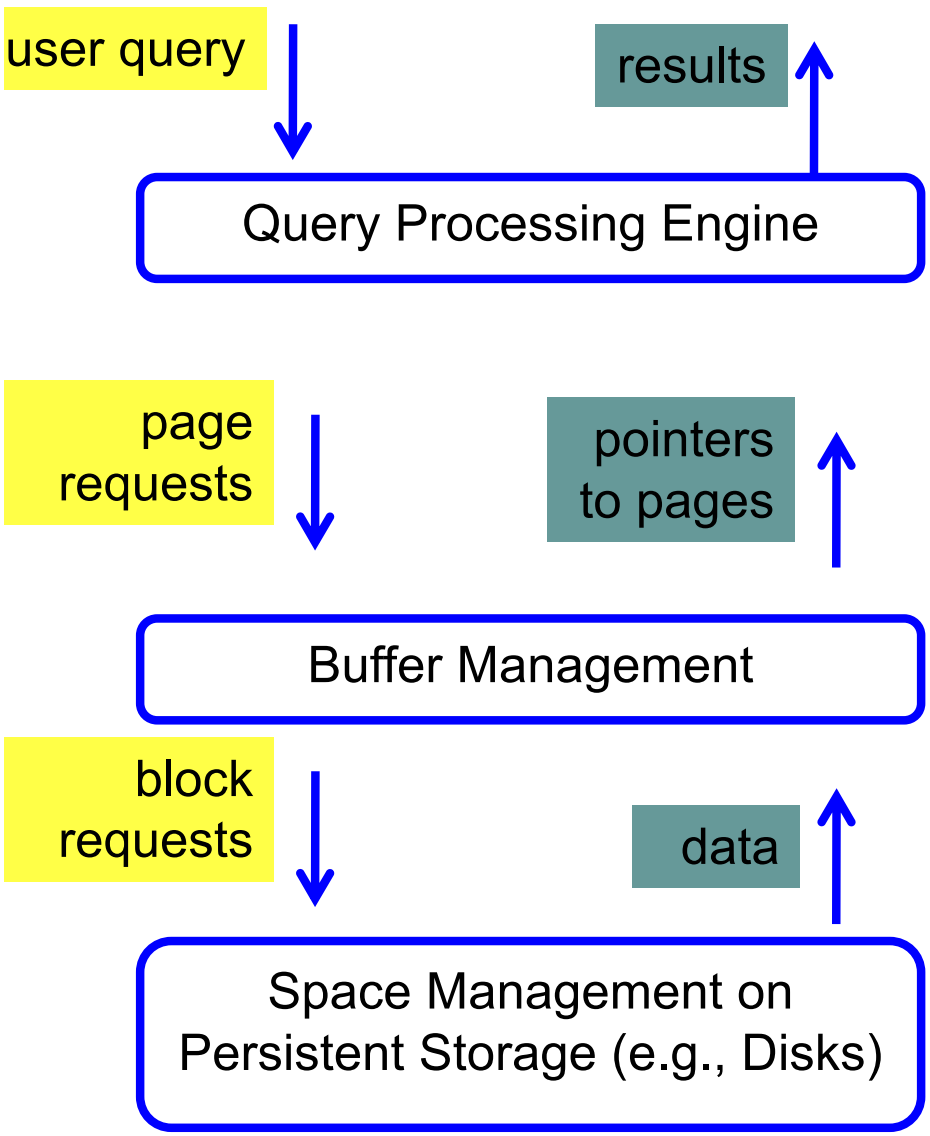
We will use PostgreSQL as an example throughout (also Programming Assignment 3)

Written in C, but well-structured codebase and easy to follow

Learning Goals

- Understand, draw, and reason about a query plan
- Reason about pipelining vs materialization for chaining of operators, and what makes sense under what conditions
- Understand how typical operators like joins, aggregates, etc., are implemented, both in centralized systems as well as parallel/distributed systems
- Design new algorithms or implementations of standard or new relational operators
- Reason about “costs” of different operators under different computing environments
- Understand query optimization issues, including equivalence between different plans

Query Processing/Storage

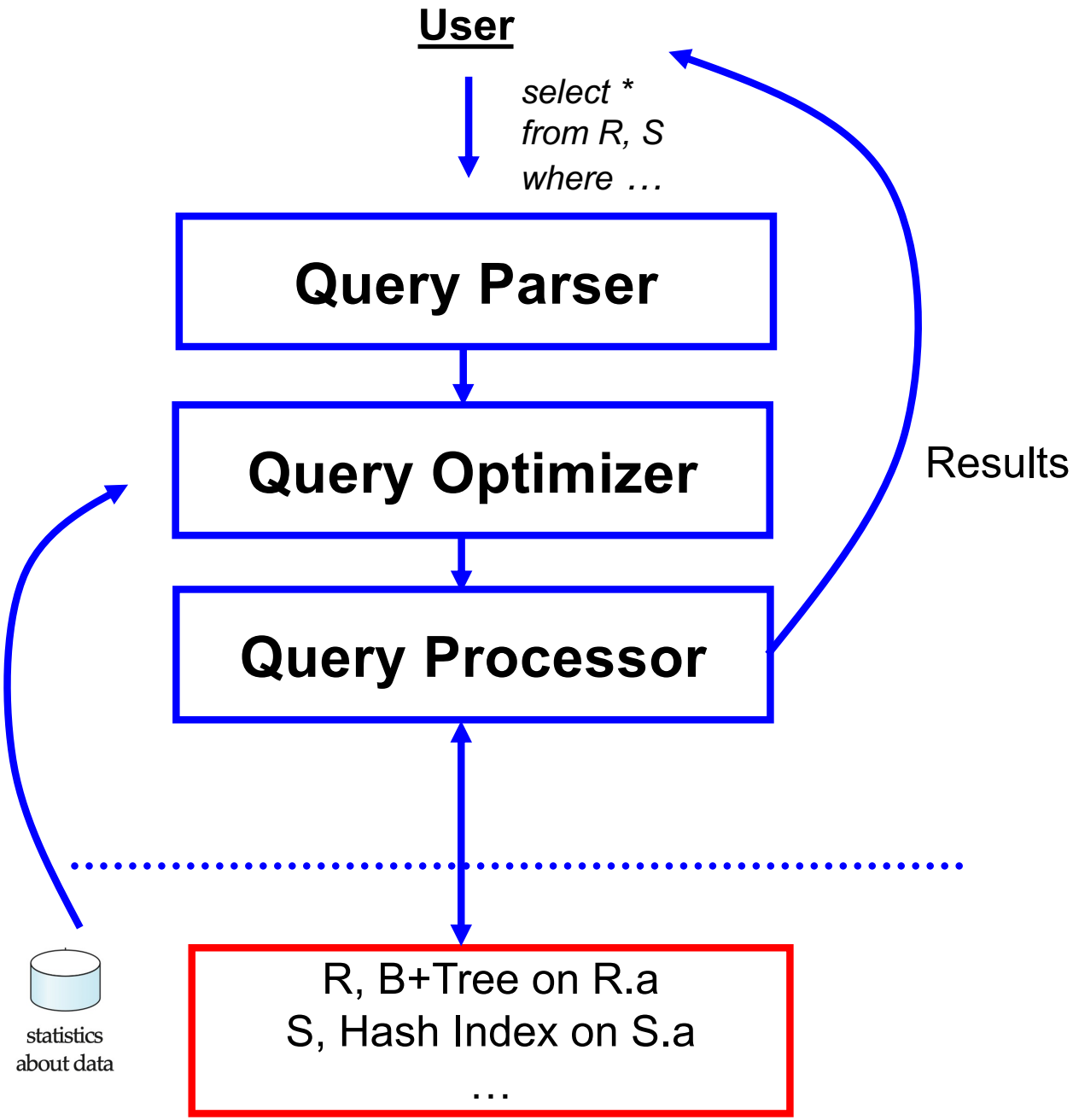


- Given an input user query, decide how to "execute" it
- Specify sequence of pages to be brought in memory
- Operate upon the tuples to produce results

- Bringing pages from disk to memory
- Managing the limited memory

- Storage hierarchy
- How are relations mapped to files?
- How are tuples mapped to disk blocks?

Getting Deeper into Query Processing



Resolve the references,
Syntax errors etc.
Converts the query to an internal format
relational algebra like

Find the *best* way to evaluate the query
Which index to use ?
What join method to use ?
…

Read the data from the files
Do the query processing
joins, selections, aggregates
…

CMSC424: Database Design

Module: Query Processing and Optimization

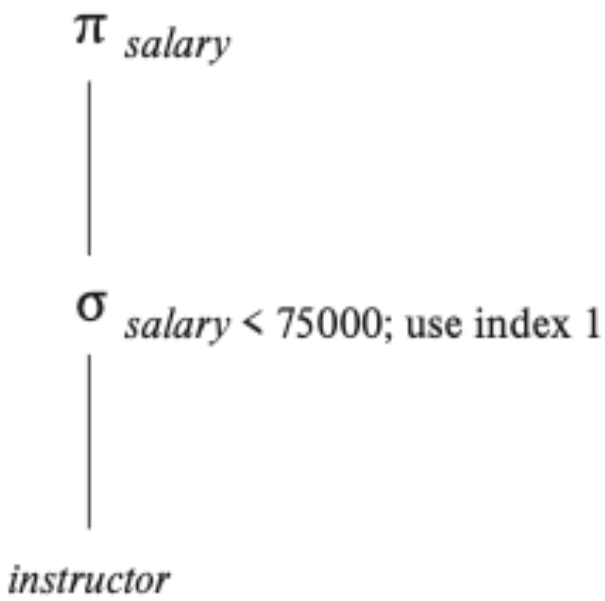
Query Plans

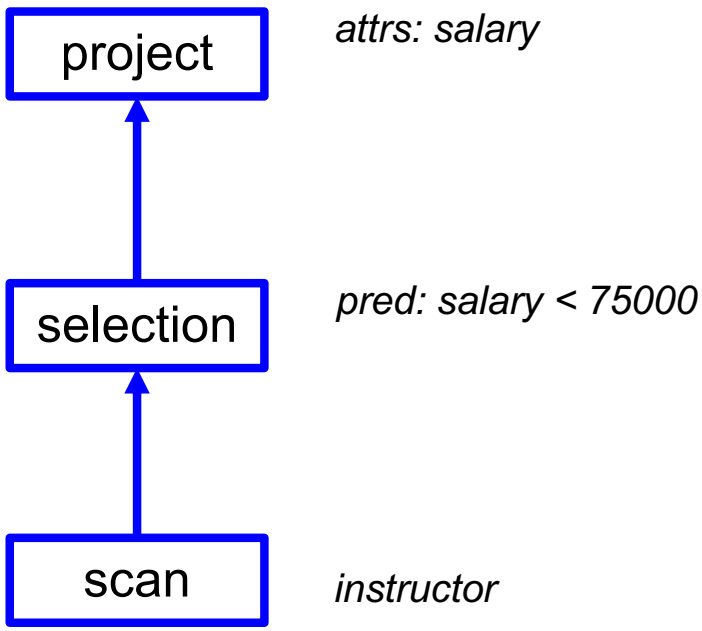
Instructor: Amol Deshpande
amol@umd.edu

Query Plan

- A tree of operators instantiated for a query
  - … with varying level of detail depending on the stage
  - … logical vs physical operators depending on the stage

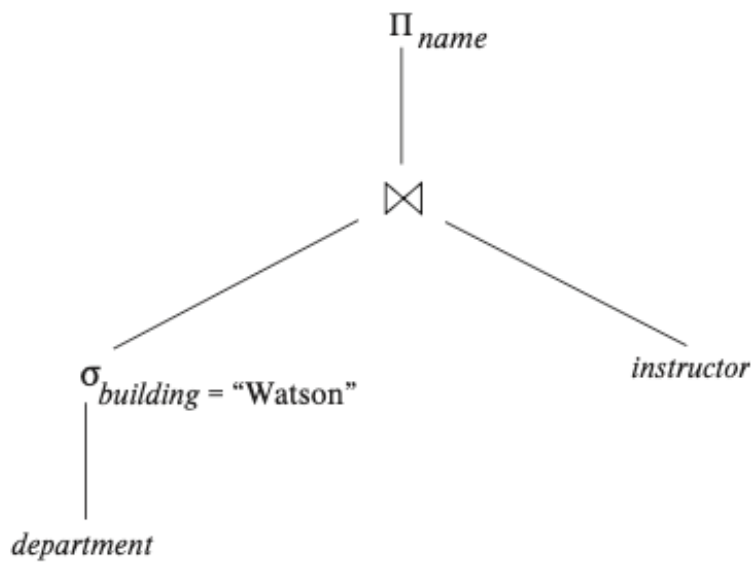
select salary
from instructor
where salary < 75000



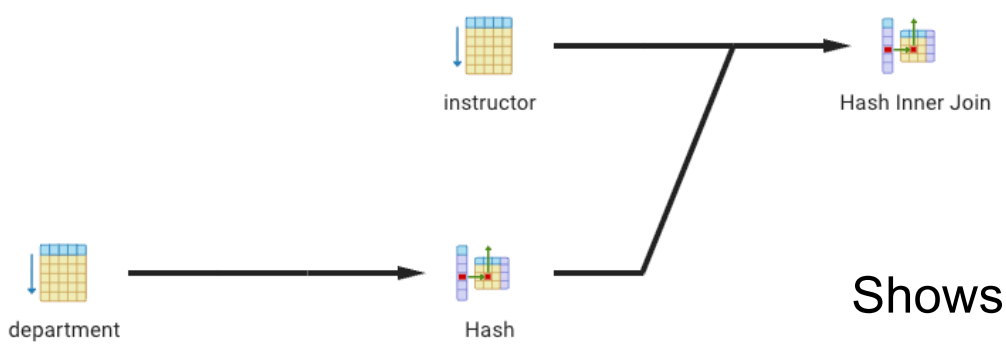


Query Plan

select name
from department natural join instructor
where building = “Watson”



A PostgreSQL execution plan



Shows “physical” operators
e.g., no explicit σ (select) or π (project)

Physical vs Logical Operators

- Logical operators: Select, Project, Natural Join, Outerjoin, Aggregate, etc.
  - Match the query language or the abstractions

- Implementation driven by other factors, especially code modularity, reuse, etc.
  - e.g., ”filter” or ”project” don’t really need to be separate physical operators – too simple for that
  - Each join variants (inner join, outer join, semi-join) doesn’t need to be a separate operator – same code can handle them
  - ”Sorting” is used by different logical operators → Should be a separate operator reused by all

- → logical operators don’t map one-to-one with physical operators

Query Plan Equivalence

- Different ways to evaluate the same query + Different ways to evaluate the same operator ➔ Need for "query optimization"

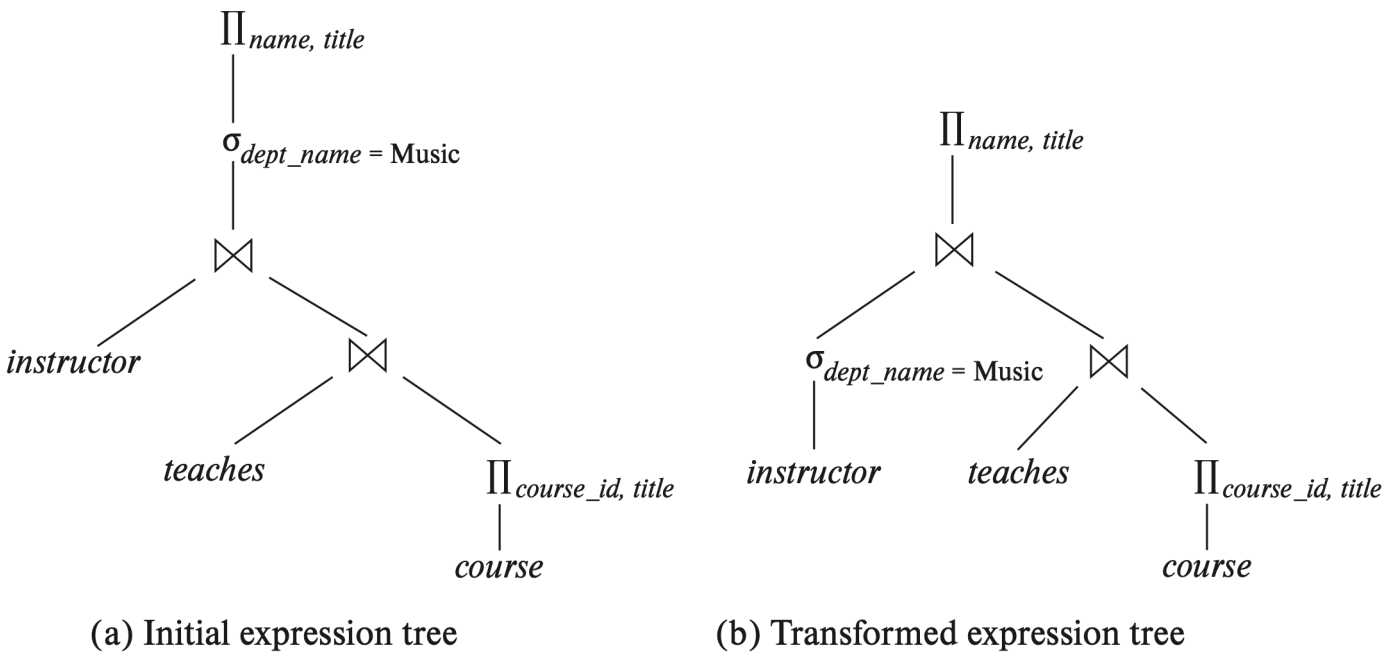
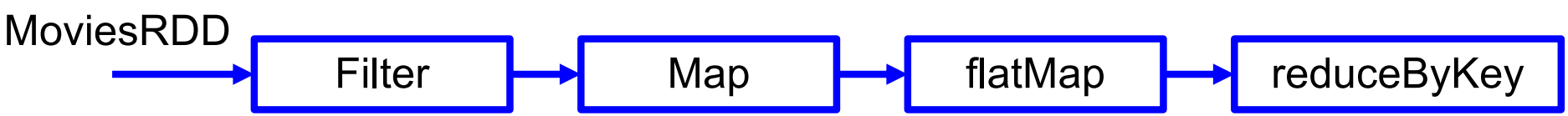


Figure 16.1 Equivalent expressions.

Spark or MongoDB Plans?

```
t3 = moviesRDD.filter(lambda t: re.search(r'\((\d{4})\)', t[1]) is not None) \
     .map(lambda t: (re.search(r'\((\d{4})\)', t[1]).group(1), t[2])) \
     .flatMap(lambda t: [((t[0], x), 1) for x in t[1].split('|')]) \
     .reduceByKey(lambda a, b: a + b)
```



- Natural mapping from the query language to the operator tree
- Less scope for “optimizations” across the operators
  - i.e., none to very few options to rearrange the operators
  - primarily because of the python functions (treated as black-boxes)
- Each operator itself needs to be optimized

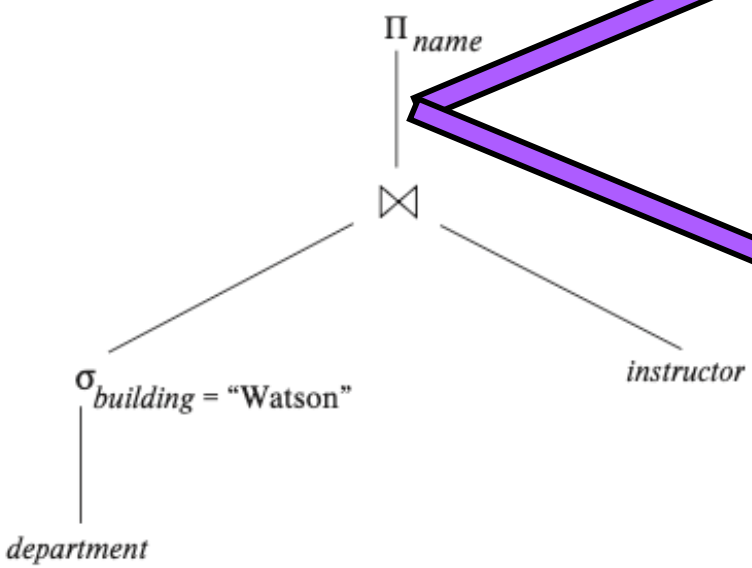
CMSC424: Database Design

Module: Query Processing and Optimization

Pipelining vs Materialization

Instructor: Amol Deshpande
amol@umd.edu

How to Pass Tuples



Materialize?

- Write out the results to a file, and next operator reads it from the file

VS

Pipeline?

- Both operators running at the same time
- Outputs from one directly passed to the other

Materialization

- Materialized evaluation is always applicable
- Cost of writing results to disk and reading them back can be quite high
- Naturally fault-tolerant
- Low memory requirements
- Often the first implementation

Pipelining

- Evaluate several operations simultaneously, passing the results of one operation on to the next.
- Cheaper: no need to store a temporary relation to disk.
- Requires higher amount of memory
  - All operations are executing at the same time (say as processes)
- Somewhat limited applicability

- A “blocking” operation: An operation that has to consume entire input before it starts producing output tuples

Operator Implementations

- Materialization
  - Relatively easy – each operator implemented to read from files and write to files
  - Pseudocode for a simple “join” operator: read from two files R and S, and write output to Out

```
open R
open Out for writing

for each tuple r in R:
    open S
    for each tuple s in S:
        if join_match(r, s):
            write joined_tuple(r, s) to Out
    close S

close R
close Out
```

Operator Implementations

- Demand-driven Pipelining using “iterator” model
  - Used in systems like PostgreSQL
- Each operator implements three functions: Open(), Next(), Close()

A ”select” operator

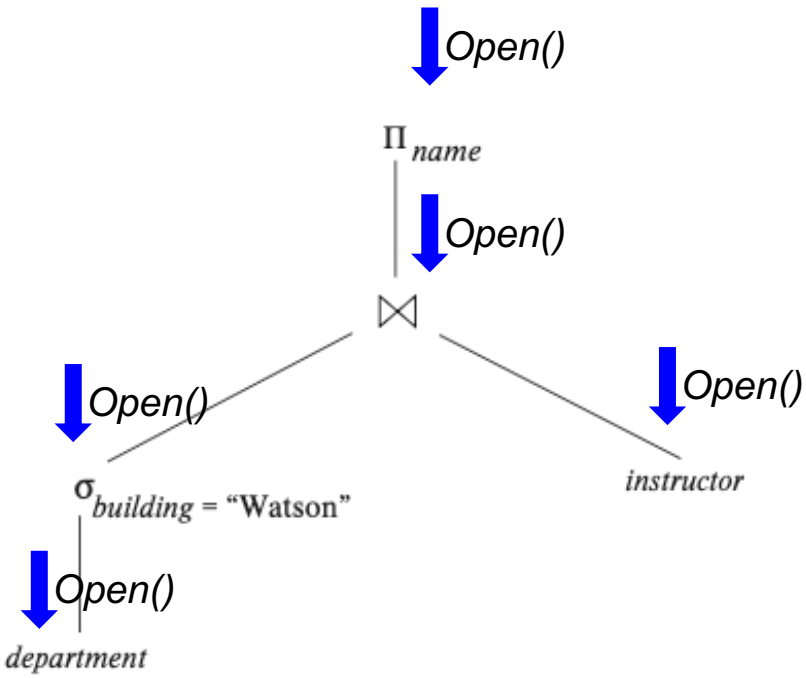
```
Select.open():
    Input.open()

Select.next():
    while True:
        t = Input.next()

        if t == EOF:
            return EOF

        if predicate(t):
            return t

Select.close():
    Input.close()
```



Operator Implementations

- Demand-driven Pipelining using “iterator” model
  - Used in systems like PostgreSQL
- Each operator implements three functions: Open(), Next(), Close()

A ”select” operator

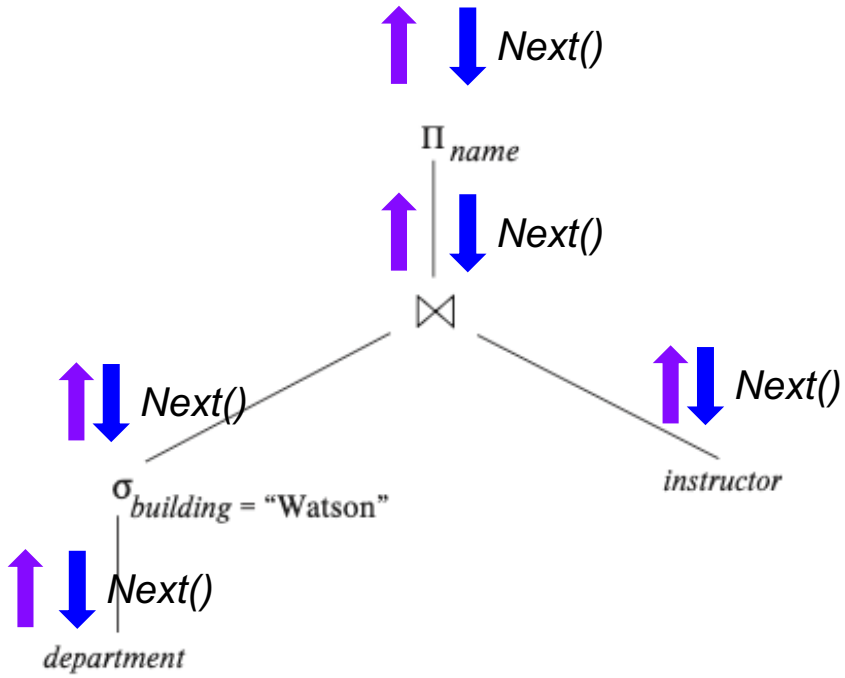
```
Select.open():
    Input.open()

Select.next():
    while True:
        t = Input.next()

        if t == EOF:
            return EOF

        if predicate(t):
            return t

Select.close():
    Input.close()
```



Operator Implementations

- Producer-drive Pipelining
  - Operators read and write to queues
  - Need to be careful in handling overfull queues
  - Easy to handle parallel setups → the queues can span machines

```
Join.run():
    r = RQ.get()
    while r != EOF:
        s = SQ.get()

        while s != EOF:
            if join_match(r, s):
                OutQ.put(join_tuple(r, s))
            s = SQ.get()

        SQ.reset()        # start S again for next r
        r = RQ.get()

    OutQ.put(EOF)
```

“Cost”

- Complicated to compute, but very important to decide early on
  - Need to know what you are “optimizing” for
- Many competing factors in today’s computing environment
  - CPU Instructions
  - Disk I/Os
  - Network Usage – either peak or average (for distributed settings)
  - Memory Usage
  - Cache Misses
  - … and so on
- Want to pick the one (or combination) that’s actually a bottleneck
  - No sense in optimizing for “memory usage” if you have a TB of memory and a single disk
  - Can do combinations by doing a weighted sum: e.g., 10 * Memory + 50 * Disk I/Os

“Cost”

- We will focus on counting the amount of data processed (in terms of disk blocks), and the CPU costs if appropriate
  - Somewhat of a simplification, but works okay
  - In particular, it doesn’t account for random vs sequential access, or the tuple layouts (mostly)

- Textbook uses a cost function that considers random vs sequential disk I/Os

CMSC424: Database Design

Module: Query Processing and Optimization

Operator Implementations

Instructor: Amol Deshpande
amol@umd.edu

Selection Operation

select * from person where SSN = “123”

```
def open(self):
    self.tableScan = TableScan("person")
    self.tableScan.open()

def next(self):
    while True:
        t = self.tableScan.next()

        if t is EOF:
            return EOF

        if t.SSN == "123":
            return t

def close(self):
    self.tableScan.close()
```

These would be initialized based on the query

Selection Operation

```
def open(self, tablename, pos, op, const):
    self.tableScan = TableScan(tablename)
    self.tableScan.open()

def next(self):
    while True:
        t = self.tableScan.next()

        if t is EOF:
            return EOF

        if evaluate(op, t[pos], const):
            return t

def close(self):
    self.tableScan.close()
```

A helper function that can handle different operators like =, <=, >=,!=, etc., and also different datatypes (e.g., integers, strings)

Cost?

 # blocks read = b_tablename

CPU cost = O(|table_name|)

Selection: B+-Tree

select * from person where SSN = “123”

```
def open(self):
        self.indexScan = BTreeLookup(index="person.SSN", key="123")
        self.indexScan.open()

def next(self):
        rid = self.indexScan.next()

        if rid is EOF:
                return EOF

        t = FetchRecord("person", rid)
        return t

def close(self):
        self.indexScan.close()
```

Cost?

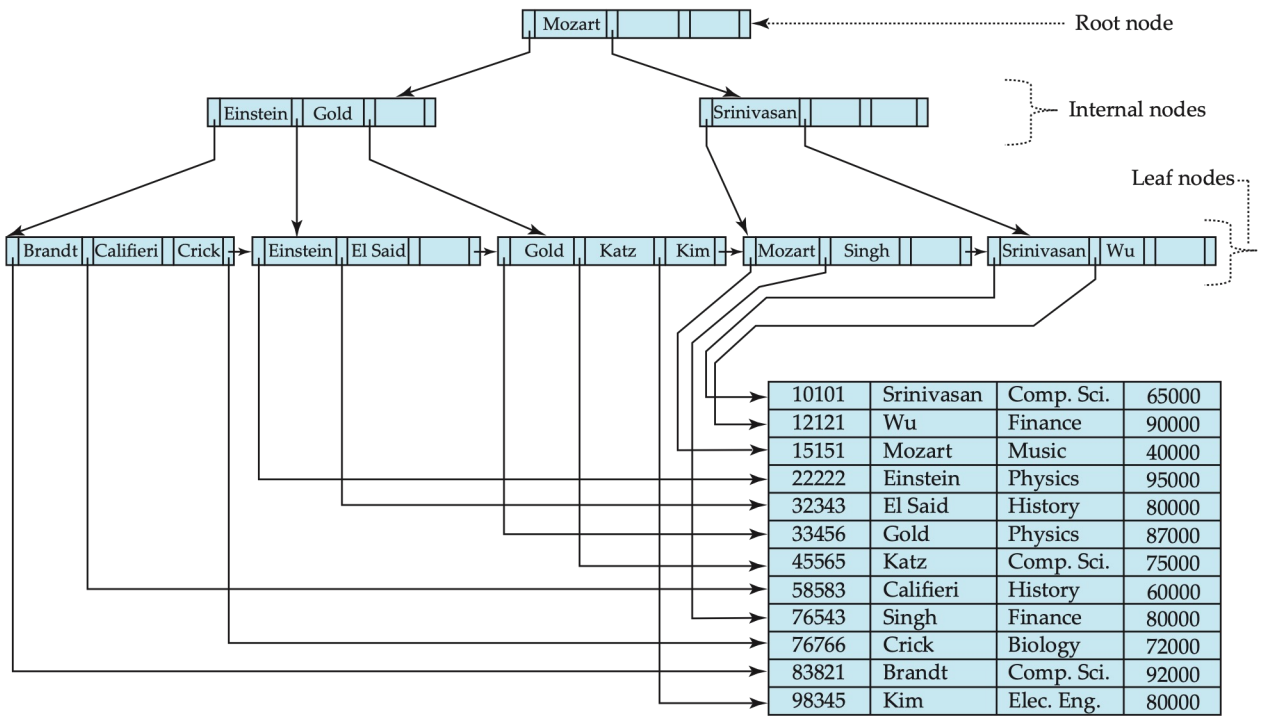
 # blocks read = h_index (height of the index) + 1

CPU cost = O(h_index + 1)

Selection Operation

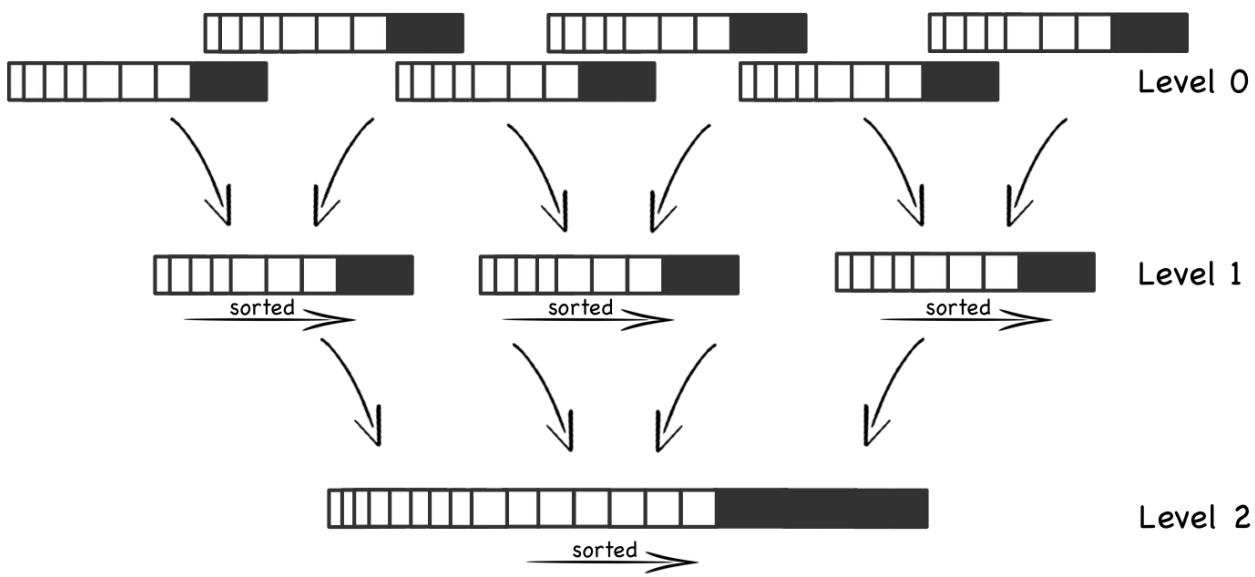
- For secondary indexes, each “FetchRecord” requires a block access
  - Can get expensive
  - You should be able to reason through a data structure and estimate the costs of such an operation

select * from instructor where name = “Singh”



Selection Operation

- Each index structure needs separate code as well as estimates
- e.g., LSMT – need to estimate the possibility to having to read the runs on disks



Compaction continues creating fewer, larger and larger files

Selection Operation

- Selections involving ranges
  - *select * from accounts where balance > 100000*
  - *select * from matches where matchdate between '10/20/06' and '10/30/06'*
  - Option 1: Sequential scan
  - Option 2: Using an appropriate index
    - Can't use hash indexes for this purpose

- Multi-attribute selections (a = 1 and b = 2)
  - Different ways to execute, including "index-ANDing" (does not require a separate operator)

Joins

- *select * from R, S where R.a = S.a*
  - Called an "*equi-join*"
- *select * from R, S where |R.a – S.a | < 0.5*
  - *Not an "equi-join"*

- *Goal: For each tuple r in R, find all "matching" tuples in S (or vice versa)*

- Simplest Algorithm ("nested loops" join)

  for each tuple r in R

  for each tuple s in S

  check if r.a = s.a (or whether |r.a – s.a| < 0.5)

- Complexity too high -- $O(N_r N_s)$
  - e.g., imagine if |R| and |S| both in millions of tuples

Index Nested-loops Join

- *select * from R, S where R.a = S.a*
  - Called an "*equi-join*"

- Let's say there is an "index" on S.a

  for each tuple r in R

  use the index to find S tuples with S.a = r.a

- Works pretty well if the index on S is a "primary" index
  - Otherwise the disk I/Os may become too much

Index Nested-loops Join

- Restricted applicability
  - An appropriate index must exist
  - What about $|R.a - S.a| < 5$?

- Great for queries with joins and selections

  *select **

  from accounts, customers

  where accounts.customer-SSN = customers.customer-SSN and

  accounts.acct-number = "A-101"

- Only need to access one SSN from the other relation

Hash Join

- <u>Assume S is the smaller relation and fits in memory</u>

  read S in memory and build a hash index (hash table) on it

  for each tuple r in R

  use the hash index on S to find tuples such that S.a = r.a

- Cost: O(N_r + N_s + *O) -- where O is the number of output tuples*

- Why good ?
  - CPU cost is much better

Hash Join

```
def open(self):
    self.H = build_hash_table(on="S", key="a")
    self.rScan = TableScan("R")
    self.rScan.open()
    self.matches = []
    self.r = None

def next(self):
    while True:
        if self.matches is not empty:
            s = self.matches.pop()
        return join(self.r, s)

        self.r = self.rScan.next()
        if self.r is EOF:
            return EOF

        self.matches = self.H.lookup(self.r.a)

def close(self):
    self.rScan.close()
```

Hash table on S built here
Usually it is built in the first call to "next"
principle: don't touch data in open()

For each tuple in r, get all matches here, but return them one-by-one

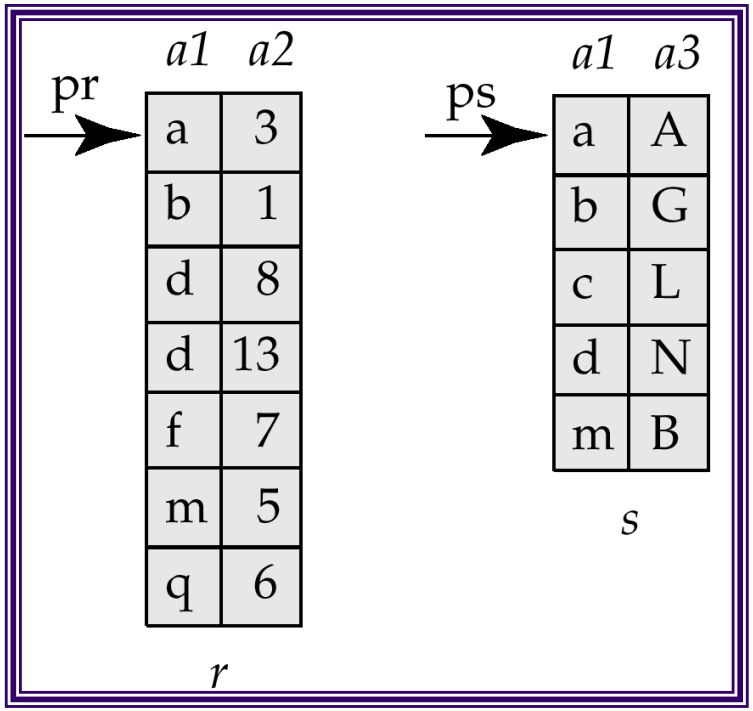
Merge-Join (Sort-merge join)

- Pre-condition:
  - The relations must be sorted by the join attribute
  - If not sorted, can sort first, and then use this algorithms
- Called “sort-merge join” sometimes

*select **
from r, s
where r.a1 = s.a1

Step:

1. *Compare the tuples at pr and ps*
2. *Move pointers down the list*
   - *Depending on the join condition*
3. *Repeat*



Merge-Join (Sort-merge join)

- Cost:
  - If the relations sorted, then just
    - $b_r + b_s$ block transfers, some seeks depending on memory size
  - What if not sorted ?
    - Then sort the relations first
    - In many cases, still very good performance
    - Typically comparable to hash join
- Observation:
  - The final join result will also be sorted on *a1*
  - This might make further operations easier to do
    - E.g. duplicate elimination

Joins: Summary

- Nested-loops join
  - Can always be applied irrespective of the join condition
  - e.g., |R.a – S.a| < 1
- Index Nested-loops join
  - Only applies if an appropriate index exists
- Hash joins – only for equi-joins
  - Join algorithm of choice when the relations are large
- Sort-merge join
  - Very commonly used – especially since relations are typically sorted
  - Sorted results commonly desired at the output
    - To answer group by queries, for duplicate elimination, because of ASC/DSC

Group By and Aggregation

select a, count(b)
from R
group by a;

- Hash-based algorithm
- Steps:
  - Create a hash table on *a,* and keep the *count(b) so far*
  - Read *R* tuples one by one
  - For a new *R tuple, "r"*
    - Check if *r.a* exists in the hash table
    - If yes, increment the count
    - If not, insert a new value

Group By and Aggregation

```
def open(self):
    self.rScan = TableScan("R")
    self.rScan.open()
    self.H = HashTable()
    self.groups = None

def next(self):
    // FIRST CALL
    if self.groups is None:
        while True:
            r = self.rScan.next()
            if r is EOF:
                break
            self.H[r.a] = self.H.get(r.a, 0) + 1

        self.groups = self.H.entries()

    // FIRST AND LATER CALLS
    if self.groups is empty:
        return EOF

    return self.groups.pop()

def close(self):
    self.rScan.close()
```

End result: H[r.a] = count

Group By and Aggregation

```
def open(self):
    self.rScan = TableScan("R")
    self.rScan.open()
    self.H = HashTable()
    self.groups = None

def next(self):
    // FIRST CALL
    if self.groups is None:
        while True:
            r = self.rScan.next()
            if r is EOF:
                break
            self.H[r.a] = self.H.get(r.a, 0) + 1

        self.groups = self.H.entries()

    // FIRST AND LATER CALLS
    if self.groups is empty:
        return EOF

    return self.groups.pop()

def close(self):
    self.rScan.close()
```

SUM(B)
self.H[r.a] = self.H.get(r.a, 0) + r.B

MIN(B)
self.H[r.a] = min(self.H.get(r.a, INFINITY), r.B)

MAX(B)
self.H[r.a] = max(self.H.get(r.a, -INFINITY), r.B)

How about MEDIAN(B)?
self.H[r.a] = self.H.get(r.a, []).append(B)

e = self.groups.pop()
return (e[0], median(e[1]))

Group By and Aggregation

select a, count(b)
from R
group by a;

- Sort-based algorithm
- Steps:
  - Sort *R* on *a*
  - Now all tuples in a single group are contigous
  - Read tuples of *R (sorted)* one by one and compute the aggregates

Group By and Aggregation

select a, AGGR(b) from R group by a;

- sum(), count(), min(), max(): only need to maintain one value per group
  - Called “distributive”
- average() : need to maintain the “sum” and “count” per group
  - Called “algebraic”
- stddev(): algebraic, but need to maintain some more state
- median(): can do efficiently with sort, but need two passes (called “holistic”)
  - First to find the number of tuples in each group, and then to find the median tuple in each group
- count(distinct b): must do duplicate elimination before the count

Duplicate Elimination

select distinct a
from R ;

- Best done using sorting – Can also be done using hashing
- Steps:
  - Sort the relation *R*
  - Read tuples of *R* in sorted order
  - prev = null;
  - for each tuple *r* in *R (sorted)*
    - if *r* != *prev* then
      - Output *r*
      - *prev* = *r*
    - else
      - Skip *r*

Set operations

*(select * from R) union (select * from S) ;*
*(select * from R) intersect (select * from S) ;*
*(select * from R) union all (select * from S) ;*
*(select * from R) intersect all (select * from S) ;*

- Remember the rules about duplicates
- "union all": just append the tuples of *R and S*
- "union": append the tuples of *R and S, and do duplicate elimination*
- *"intersection"*: similar to joins
  - Find tuples of R and S that are identical on all attributes
  - Can use *hash-based or sort-based algorithm*

Outer Joins

- Say: R FULL OUTER JOIN S, on R.a = S.a
- Need to keep track of which tuples of R "do not match" any tuples from S, and vice versa

- Hash-based, with a hash index on S:
  - For a tuple r in R, if the probe returns NULL, output r padded with NULLs
  - For each tuple *s* in *S*, maintain a Boolean variable (in the hash table) to track whether *s* was returned for any probes
  - At the end, go through the hash table, and look for *S* tuples that did not match anything

- Merge join can also be adapted in a similar way